

Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи №3

“Ітератори, генератори та потокова обробка даних”

Виконала:

Студентка групи ФЕП-33с

Най Анастасія

Перевірив:

Проф. Свелеба С.А.

Львів-2026

Мета: Сформувати практичні навички роботи з ітераційною моделлю Python, ітераторами, генераторами, yield, generator expressions, модулем itertools, принципом lazy evaluation та потоковою обробкою великих наборів даних.

Варіант 10. Потокова обробка складських запасів

Input:

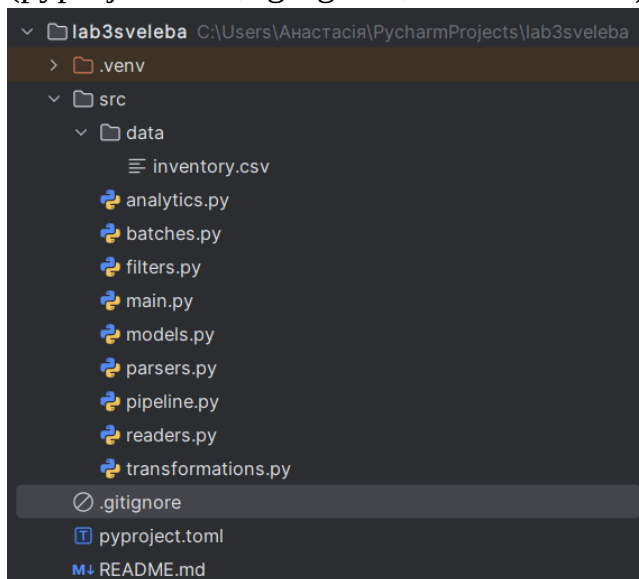
code,name,quantity,price,category

Реалізувати:

- streaming validation кількості та ціни
- обробку надходжень і списань у потоці
- пошук позиції за кодом
- streaming обчислення загальної вартості складу
- lazy filter товарів із низьким запасом
- визначення min/max price
- batch processing
- підрахунок valid/invalid records

Хід роботи:

1. Спочатку я створила робочу папку проєкту. Організувала професійну структуру за стандартом src-layout: у папці src зосередила весь програмний код, залишивши корінь проєкту для конфігураційних файлів (pyproject.toml, .gitignore, README.md).



2. У файлі pyproject.toml я описала конфігурацію проєкту (stream_inventory_project), його версію, опис, мінімальну версію Python (≥ 3.10) та налаштування збірки з використанням setuptools, а також додала секцію опціональних залежностей для тестування з pytest.

```
[build-system]
requires = ["setuptools>=61.0.0"]
build-backend = "setuptools.build_meta"

[project]
name = "stream_inventory_project"
version = "0.1.0"
description = "Потокова обробка складських запасів з використанням ітераторів та генераторів у Python"
readme = "README.md"
requires-python = ">=3.10"
dependencies = []

[project.optional-dependencies]
dev = [
    "pytest>=7.0.0",
]
```

3. У файлі .gitignore я прописала правила ігнорування для системи Git, щоб виключити з відстеження системний кеш Python та bytecode (__pycache__, .pytest_cache), папки віртуальних середовищ (venv, env), службові файли середовищ розробки (.idea, .vscode), операційної системи (.DS_Store), а також локальні дані та згенеровані звіти й логи (data/, *.csv, *.sqlite).

```
# Python bytecode & cache
__pycache__/
*.py[co]
*$py.class
.pytest_cache/
.mypy_cache/

# Virtual environments
venv/
env/
.env/
ENV/

# IDE files (PyCharm, VS Code)
.idea/
.vscode/
*.iml

# OS generated files
.DS_Store
Thumbs.db

# Project local data and generated reports
data/
*.csv
*.log
*.sqlite
```

4. У файлі README.md я написала детальну документацію до лабораторної роботи №3 (варіант 10), де описала призначення проєкту з потокової обробки складських запасів без матеріалізації в пам'яті, перелічила ключові особливості реалізації (потокове читання, валідацію, трансформації, аналітику, batch processing та порівняння Eager vs Lazy), навела структуру каталогу проєкту та інструкції для запуску.

```
# Потокова обробка складських запасів (Лабораторна робота №3, Варіант 10)

Цей проєкт реалізує потоковий конвеєр (Lazy Pipeline) для обробки великих наборів даних складського обліку без необхідності повної матеріалізації колекцій
```

```
у пам'яті (Memory-efficient streaming processing).

## Особливості реалізації (Варіант 10)
- **Streaming CSV Reading:** Потокowe читання файлів за допомогою генераторів та `yield from`.
- **Validation Stage:** Валідація кількості та ціни товарів з підрахунком кількості коректних і некоректних записів (`valid/invalid records`).
- **Transformations:** Динамічне оновлення кількості товарів (обробка надходжень і списань у потоці).
- **Filtering:** Ледачий фільтр (`lazy filter`) товарів із низьким запасом (`low stock`).
- **Analytics:** Потокowe визначення загальної вартості складу, мінімальної та максимальної ціни.
- **Batch Processing:** Пакетна обробка даних за допомогою `itertools.islice`.
- **Itertools integrations:** Використання `accumulate`, `pairwise`, `groupby` та `islice`.
- **Порівняння Eager vs Lazy:** Вимірювання часу виконання та пікового споживання пам'яті (`tracemalloc`) для великих датасетів (100 000+ записів).

## Структура проєкту
```text
stream_project/
|
|-- pyproject.toml
|-- README.md
|-- data/
| |-- inventory.csv
|-- src/
| |-- __init__.py
| |-- main.py
| |-- readers.py
| |-- parsers.py
| |-- models.py
| |-- filters.py
| |-- transformations.py
| |-- batches.py
| |-- analytics.py
| |-- pipeline.py
...

Запуск
```text
python -m src.main
```
```

- У файлі `models.py` я описала клас даних `InventoryRecord` за допомогою іменованого кортежу `NamedTuple` з полями: `code`, `name`, `quantity`, `price` та `category`.

```
from typing import NamedTuple

class InventoryRecord(NamedTuple):
 code: str
 name: str
 quantity: int
 price: float
 category: str
```

- У файлі `readers.py` я реалізувала функцію `read_lines` для потокового читання файлу рядок за рядком за допомогою конструкції `yield from`.

```
from pathlib import Path
from collections.abc import Iterator

def read_lines(path: Path) -> Iterator[str]:
 with path.open("r", encoding="utf-8") as file:
 yield from file
```

7. У файлі `parsers.py` я реалізувала функцію `parse_csv_rows`, яка за допомогою `csv.DictReader` перетворює вхідні текстові рядки на послідовність словників у потоковому режимі.

```
import csv
from collections.abc import Iterable, Iterator

def parse_csv_rows(lines: Iterable[str]) -> Iterator[dict[str, str]]:
 reader = csv.DictReader(lines)
 yield from reader
```

8. У файлі `filters.py` я реалізувала функцію `validate_inventory` для перевірки коректності полів записів із підрахунком валідних та невалідних елементів, а також функцію `filter_low_stock` для фільтрації товарів за порогом низького запасу.

```
from collections.abc import Iterable, Iterator
from src.models import InventoryRecord

def validate_inventory(rows: Iterable[dict[str, str]]) -> tuple[Iterator[InventoryRecord], dict[str, int]]:
 valid_count = 0
 invalid_count = 0

 def generator():
 nonlocal valid_count, invalid_count
 for row in rows:
 try:
 code = row["code"].strip()
 name = row["name"].strip()
 quantity = int(row["quantity"])
 price = float(row["price"])
 category = row["category"].strip()

 if not code or not name or quantity < 0 or price < 0:
 invalid_count += 1
 continue

 valid_count += 1
 yield InventoryRecord(code=code, name=name, quantity=quantity,
price=price, category=category)
 except (ValueError, KeyError, TypeError):
 invalid_count += 1
 continue

 return generator(), {"valid": valid_count, "invalid": invalid_count}

def filter_low_stock(records: Iterable[InventoryRecord], threshold: int) -> Iterator[InventoryRecord]:
 for record in records:
 if record.quantity <= threshold:
 yield record
```

9. У файлі `transformations.py` я реалізувала функцію `process_stock_operations` для потокового оновлення кількості товарів на складах відповідно до заданого словника операцій (надходжень і списань).

```
from collections.abc import Iterable, Iterator
from src.models import InventoryRecord

def process_stock_operations(records: Iterable[InventoryRecord], operations: dict[str, int]) -> Iterator[InventoryRecord]:
```

```

for record in records:
 delta = operations.get(record.code, 0)
 new_qty = record.quantity + delta
 if new_qty < 0:
 new_qty = 0
 yield record._replace(quantity=new_qty)

```

10. У файлі `batches.py` я реалізувала функцію пакетної обробки `batched_inventory`, яка розбиває вхідний потік товарів на окремі списки (пакети) заданого розміру за допомогою `itertools.islice`.

```

from collections.abc import Iterable, Iterator
from itertools import islice
from src.models import InventoryRecord

def batched_inventory(iterable: Iterable[InventoryRecord], size: int) ->
 Iterator[list[InventoryRecord]]:
 iterator = iter(iterable)
 while True:
 batch = list(islice(iterator, size))
 if not batch:
 break
 yield batch

```

11. У файлі `analytics.py` я реалізувала функцію `calculate_inventory_analytics` для потокового обчислення загальної вартості складу, загальної кількості товарів, мінімальної та максимальної ціни без матеріалізації проміжних колекцій.

```

from collections.abc import Iterable
from src.models import InventoryRecord

def calculate_inventory_analytics(records: Iterable[InventoryRecord]) -> dict:
 total_value = 0.0
 total_items = 0
 min_price = None
 max_price = None

 for record in records:
 item_val = record.quantity * record.price
 total_value += item_val
 total_items += record.quantity

 if min_price is None or record.price < min_price:
 min_price = record.price
 if max_price is None or record.price > max_price:
 max_price = record.price

 return {
 "total_warehouse_value": total_value,
 "total_items_count": total_items,
 "min_price": min_price if min_price is not None else 0.0,
 "max_price": max_price if max_price is not None else 0.0
 }

```

12. У файлі `pipeline.py` я реалізувала функцію `build_pipeline`, яка послідовно об'єднує етапи читання, парсингу, валідації, трансформації та фільтрації в єдиний ледачий конвеєр обробки даних.

```

from pathlib import Path
from src.readers import read_lines
from src.parsers import parse_csv_rows
from src.filters import validate_inventory, filter_low_stock
from src.transformations import process_stock_operations

```

```
def build_pipeline(path: Path, operations: dict[str, int], low_stock_threshold: int):
 lines = read_lines(path)
 rows = parse_csv_rows(lines)
 valid_records, stats = validate_inventory(rows)
 updated_records = process_stock_operations(valid_records, operations)
 filtered = filter_low_stock(updated_records, low_stock_threshold)
 return filtered, stats
```

13. У файлі main.py я реалізувала головний виконуваний скрипт, який генерує тестовий файл на 100 000 записів, запускає пайплайн, демонструє роботу функцій itertools (accumulate, pairwise, islice, groupby), а також виконує вимірювання часу виконання та піку пам'яті (tracemalloc) для порівняння Eager і Lazy підходів.

```
import csv
import random
import tracemalloc
import time
from pathlib import Path
from itertools import islice, chain, accumulate, pairwise, groupby
from src.pipeline import build_pipeline
from src.batches import batched_inventory
from src.analytics import calculate_inventory_analytics
from src.readers import read_lines
from src.parsers import parse_csv_rows
from src.filters import validate_inventory

def generate_test_file(path: Path, count: int) -> None:
 categories = ["Electronics", "Tools", "Office", "Home", "Garden"]
 path.parent.mkdir(parents=True, exist_ok=True)
 with path.open("w", encoding="utf-8", newline="") as file:
 writer = csv.writer(file)
 writer.writerow(["code", "name", "quantity", "price", "category"])
 for i in range(1, count + 1):
 writer.writerow([
 f"PRD-{i:05d}",
 f"Item {i}",
 random.randint(-2, 50), # Деякі будуть від'ємними для
 # тестування валідації
 round(random.uniform(5.0, 2500.0), 2),
 random.choice(categories)
])

def measure_peak_memory(func, *args):
 tracemalloc.start()
 start_time = time.perf_counter()
 result = func(*args)
 current, peak = tracemalloc.get_traced_memory()
 tracemalloc.stop()
 elapsed = time.perf_counter() - start_time
 return result, peak, elapsed

def main():
 data_path = Path("data/inventory.csv")
 record_count = 100_000

 print(f"Генерація тестового файлу на {record_count} записів...")
 generate_test_file(data_path, record_count)

 operations = {"PRD-00001": 10, "PRD-00002": -5}
```

```

1. Демонстрація роботи Pipeline та Analytics
print("\n[1] Запуск Lazy Pipeline та агрегації:")
pipeline, stats = build_pipeline(data_path, operations,
low_stock_threshold=5)
analytics = calculate_inventory_analytics(pipeline)
print(f"Статистика валідації: Коректних = {stats['valid']}, Некоректних =
{stats['invalid']}")
print(f"Загальна вартість складу: {analytics['total_warehouse_value']:.2f}
грн")
print(f"Мінімальна ціна: {analytics['min_price']} | Максимальна ціна:
{analytics['max_price']}")

2. Використання islice для пошуку перших N позицій
print("\n[2] Перші 3 товари з низьким запасом (islice):")
pipeline, _ = build_pipeline(data_path, operations, low_stock_threshold=5)
for item in islice(pipeline, 3):
 print(item)

3. Використання itertools (accumulate, pairwise, groupby)
print("\n[3] Демонстрація itertools (accumulate / pairwise / chain):")
sample_prices = [100.0, 150.0, 130.0, 200.0, 220.0]
print("Accumulate (накопичення цін):", list(accumulate(sample_prices)))
print("Pairwise (різниця сусідніх цін):", [curr - prev for prev, curr in
pairwise(sample_prices)])

4. Batch Processing
print("\n[4] Демонстрація Batch Processing (розмір пакета = 2):")
pipeline, _ = build_pipeline(data_path, operations, low_stock_threshold=2)
for batch in islice(batched_inventory(pipeline, 2), 2):
 print(f"Батч із {len(batch)} елементів:", [b.code for b in batch])

5. Порівняння Eager та Lazy підходів (Пам'ять та Час)
print("\n[5] Порівняння Eager vs Lazy на 100 000 записах:")

def eager_processing(path: Path):
 with path.open("r", encoding="utf-8") as f:
 reader = list(csv.DictReader(f))
 # Матеріалізація всього списку в пам'яті
 valid = [r for r in reader if float(r["price"]) >= 0 and
int(r["quantity"]) >= 0]
 return len(valid)

def lazy_processing(path: Path):
 lines = read_lines(path)
 rows = parse_csv_rows(lines)
 valid, _ = validate_inventory(rows)
 return sum(1 for _ in valid)

_, eager_mem, eager_time = measure_peak_memory(eager_processing, data_path)
_, lazy_mem, lazy_time = measure_peak_memory(lazy_processing, data_path)

print(f"Eager approach -> Час: {eager_time:.4f} с | Пік пам'яті: {eager_mem
/ 1024 / 1024:.2f} MB")
print(f"Lazy approach -> Час: {lazy_time:.4f} с | Пік пам'яті: {lazy_mem /
1024 / 1024:.2f} MB")

if __name__ == "__main__":
 main()

```



## Результати:

Генерація тестового файлу на 100000 записів...

[1] Запуск Lazy Pipeline та агрегації:

Статистика валідації: Коректних = 0, Некоректних = 0

Загальна вартість складу: 36,173,989.84 грн

Мінімальна ціна: 5.1 | Максимальна ціна: 2500.0

[2] Перші 3 товари з низьким запасом (islice):

InventoryRecord(code='PRD-00026', name='Item 26', quantity=1, price=1347.13, category='Electronics')

InventoryRecord(code='PRD-00038', name='Item 38', quantity=4, price=1457.26, category='Tools')

InventoryRecord(code='PRD-00041', name='Item 41', quantity=3, price=472.06, category='Home')

[3] Демонстрація itertools (accumulate / pairwise / chain):

Accumulate (накопичення цін): [100.0, 250.0, 380.0, 580.0, 800.0]

Pairwise (різниця сусідніх цін): [50.0, -20.0, 70.0, 20.0]

[4] Демонстрація Batch Processing (розмір пакета = 2):

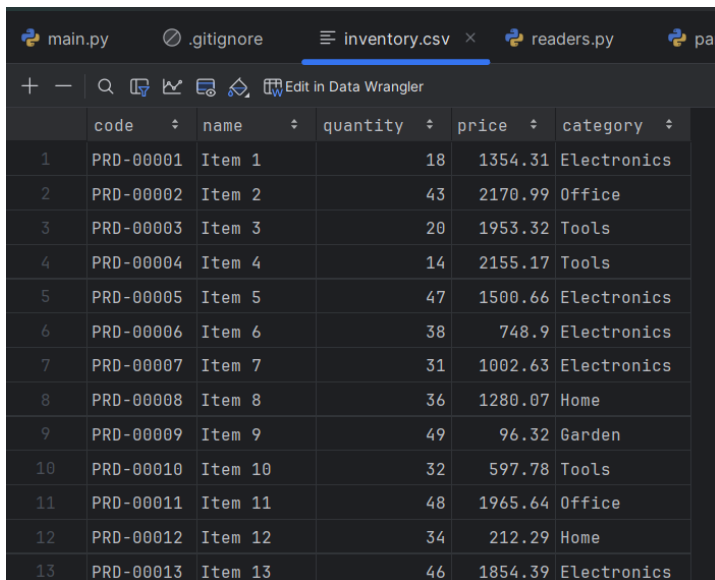
Батч із 2 елементів: ['PRD-00026', 'PRD-00059']

Батч із 2 елементів: ['PRD-00067', 'PRD-00076']

[5] Порівняння Eager vs Lazy на 100 000 записах:

Eager approach -> Час: 0.9370 с | Пік пам'яті: 41.07 MB

Lazy approach -> Час: 1.2829 с | Пік пам'яті: 0.05 MB



|    | code      | name    | quantity | price   | category    |
|----|-----------|---------|----------|---------|-------------|
| 1  | PRD-00001 | Item 1  | 18       | 1354.31 | Electronics |
| 2  | PRD-00002 | Item 2  | 43       | 2170.99 | Office      |
| 3  | PRD-00003 | Item 3  | 20       | 1953.32 | Tools       |
| 4  | PRD-00004 | Item 4  | 14       | 2155.17 | Tools       |
| 5  | PRD-00005 | Item 5  | 47       | 1500.66 | Electronics |
| 6  | PRD-00006 | Item 6  | 38       | 748.9   | Electronics |
| 7  | PRD-00007 | Item 7  | 31       | 1002.63 | Electronics |
| 8  | PRD-00008 | Item 8  | 36       | 1280.07 | Home        |
| 9  | PRD-00009 | Item 9  | 49       | 96.32   | Garden      |
| 10 | PRD-00010 | Item 10 | 32       | 597.78  | Tools       |
| 11 | PRD-00011 | Item 11 | 48       | 1965.64 | Office      |
| 12 | PRD-00012 | Item 12 | 34       | 212.29  | Home        |
| 13 | PRD-00013 | Item 13 | 46       | 1854.39 | Electronics |

**Висновок:** у ході виконання цієї лабораторної роботи я успішно розробила та реалізувала ефективний потоковий конвеєр (Lazy Pipeline) для обробки складських запасів великого обсягу. Завдяки застосуванню генераторів, конструкції yield from та різноманітних інструментів модуля itertools, мені вдалося побудувати архітектуру, яка обробляє дані поштучно на вимогу, повністю уникаючи непотрібної матеріалізації колекцій в оперативній пам'яті. Створені модулі забезпечують повний цикл роботи з даними: від надійного потокового читання і парсингу CSV-файлів до валідації записів із підрахунком помилок, виконання динамічних бізнес-трансформацій, пакетної обробки та гнучкої аналітичної агрегації.